Creating Class and Object

Lecture 7

Based on Slides of Dr. Norazah Yusof

Classes

A Java class member may have:

- **instance variables** to define data fields/attributes and

- **instance methods** to define behaviors.

Additionally, a class provides a special type of methods, known as **constructors**, which are invoked to construct objects from the class.

UML Class Diagram for Employee

Employee
- empnum: int - empname: String
+ setEmpNum(int): void + getEmpNum(): int + setEmpName(String): void + getEmpName(): String

Class name

Attributes / instance variables that define the object's state; can hold numbers, characters, strings, other objects

Actions that an object of this class can take (behaviors)

Data Fields

- Variables declared within class
  - But outside of any method
- Called instance variables (local)
  - When not `static` (global)
- Each class will have own copy of each non-static data field
- `private` access for fields
  - No other classes can access field's values
  - Only methods of same class allowed to use `private` variables

Data Fields (cont'd)

You name it!

private [static] [final] *datatype* *name*;

Usually private

May be present: means the field is shared by all objects in the class

May be present: means the field is a constant

primitive: **int**, **double**, etc., or an object: **String**, **Image**, **Foot**

```
private int empnum;
```

Creating Instance Methods in a Class

- **Classes contain methods**
- **Non-static method**
  - **Instance methods**
  - **“Belong” to objects**
- **Typically declare non-static instance variables**
- **`static` class variables**
  - **Are not instance variables**

7

Method Definitions

- **Method header form:**

  ***modifier returnType methodName* ([*parameterList*])**

- **Example:**

```
public double getPrice()
public void setPrice(double aPrice)
```

- **Method names usually sound like verbs.**
- **The name of a method that returns the value of a field often starts with get:**

  `getWidth, getX`

- **The name of a method that sets the value of a field often starts with set:**

  `setLocation, setText`

Understanding Data Hiding

- Data hiding using encapsulation
  - Data fields usually **`private`**
  - Client application accesses them only through **`public`** interfaces
- Set method
  - Controls data values used to set variable
- Get method
  - Controls how value retrieved

Accessor and Mutator Methods

- **The methods that retrieve the data of fields are called *accessors*.**
- **The methods that modify the data of fields are called *mutators*.**

Accessors and Mutators

- **For the rectangle example, the accessors and mutators are:**
  - **getLength: Returns the value of the length field.**

    ```
    public double getLength() { }
    ```
  - **getWidth : Returns the value of the width field.**

    ```
    public double getWidth() { }
    ```
  - **setLength: Sets the value of the length field.**

    ```
    public void setLength(double len) { }
    ```
  - **setWidth : Sets the value of the width field.**

    ```
    public void setLength(double w) { }
    ```

Class Definition

Employee
- empnum: int - empname: String
+ setEmpNum(int): void + getEmpNum(): int + setEmpName(String): void + getEmpName(): String

UML class diagram

```
public class Employee {
  private int empnum;
  private String empname;

  public void setEmpNum(int num) {
   empnum = num;
   }

  public int getEmpNum() {
    return empnum;
   }

 public void setEmpName(Strig name) {
    empname = name;
  }

 public String getEmpName() {
    return empname;
  }
}
```

Declaring Object Reference Variable and Create Object

- **To refer an object, assign the object to a reference variable.**
- **To declare a reference variable, use the syntax:**

```
ClassName objectRefVar;
```

- **To create object:**

```
objectRefVar = new ClassName();
```

Example:

```
Employee emp1;
emp1 = new Employee();
```

Declaring/Creating Objects in a Single Step

```
ClassName objectRefVar = new ClassName();
```

Assign object reference

Create an object

Example:

```
Employee emp1 = new Employee();
```

Accessing Objects

- **After object instantiated, methods accessed using:**
  - **Object's identifier**
  - **Dot**
  - **Data/Method call**
- **Referencing the object's data:**

```
objectRefVar.data
```

- **Example:**

```
emp1.empnum
```

- **Invoking the object's method:**

```
objectRefVar.methodName(arguments)
```

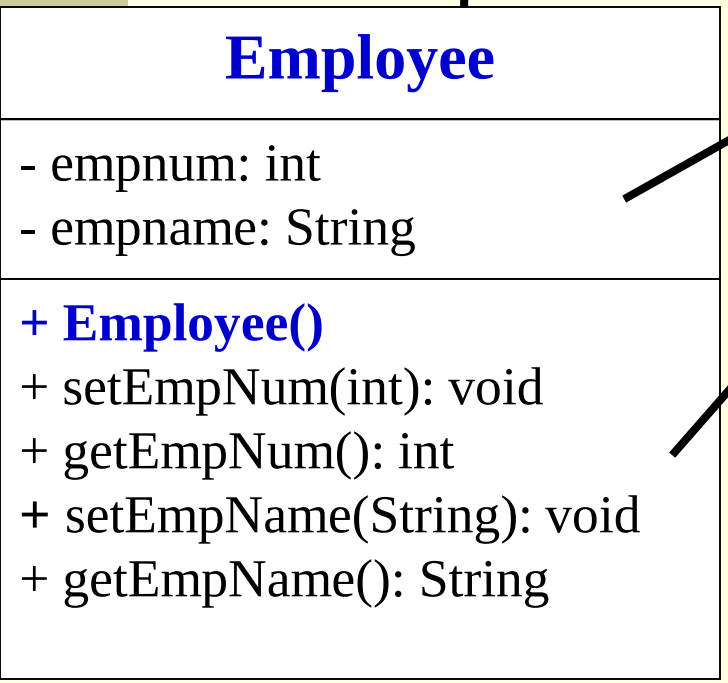
- **Example:**

```
emp1.getEmpNum()
```

Constructors

- **Special kind of methods for creating objects of the class.**
- **A class may be declared without constructors.**
- **If a programmer does not define any constructors, Java will automatically provides one default no-args constructor, which allocates memory and sets fields to the default values**
- **Name of constructor always same as name of class.**
- **Default constructors do not have a return type (not even void) and they do not return a value.**
- **Example:** `public Employee() {  }`

Class Definition with default constructor



UML class diagram

```
public class Employee {
  private int empnum;
  private String empname;

  public Employee() {
    System.out.println ("Start Employee");
  }

  public void setEmpNum(int num) {
   empnum = num;
   }
  public int getEmpNum() {
    return empnum;
   }
 public void setEmpName(String name) {
    empname = name;
  }
 public String getEmpName() {
    return empname;
  }
}
```

Constructors

- Constructors may take parameters.
  - If a constructor has one parameter, we may call it as **one-argument constructor**.
- If a class has **more than one constructor**, they must have different numbers and/or types of parameters.
  - This is called **constructor overloading**

Creating Object == invoking Constructors

- **Constructors are invoked using the new operator when an object is created. Constructors play the role of initializing objects.**
- **Format:**

```
new ClassName();
```

Example:

```
new Employee();
new Employee(123, "Ahmad");
```

Constructors (cont'd)

```
public class Fraction
{
   private int num, denom;

   public Fraction ( )
   {
      num = 0;
      denom = 1;
   }

   public Fraction (int n)
   {
      num = n;
      denom = 1;
   }
```

"No-args" constructor

Continued ⇨

```
   public Fraction (int n, int d)
   {
      num = n;
      denom = d;
      reduce ();
   }

   public Fraction (Fraction other)
   {
      num = other.num;
      denom = other.denom;
   }
   ...

}
```

Copy constructor

Constructors (cont'd)

- A nasty bug:

```
public class MyWindow
                    extends JFrame
{
   ...
   // Constructor:
   public void MyWindow ( )
   {
      ...
   }
   ...
```

Compiles fine, but the compiler thinks this is a <u>method</u> and uses **MyWindow**'s default no-args constructor instead.

Constructors (cont'd)

- Constructors of a class can call each other using the keyword **this** — a good way to avoid duplicating code:

```
public class Fraction
{
   ...

   public Fraction (int n)
   {
      this (n, 1);
   }
   ...
```

```
   ...
   public Fraction (int p, int q)
   {
      num = p;
      denom = q;
      reduce ();
   }
   ...
```

Organizing Classes

- Place **data fields** in logical order
  - At beginning of class
  - Fields listed vertically
- May place data fields and methods in any order within a class
  - Common to list all data fields first
  - Can see names and data types before reading methods that use data fields

Employee.java

import ...

import statements

public class Employee

Class header

{

- Fields

Attributes / variables that define the object's state; can hold numbers, characters, strings, other objects

- Constructors

Procedures for constructing a new object of this class and initializing its fields

- Methods

Actions that an object of this class can take (behaviors)

}

Lab Exercise 2

- **Exercise 1 - 6 (page 17 - 27)**

Passing Arguments to Methods

Lecture 8

Based on Slides of Dr. Norazah Yusof

Method

- A method is a collection of statements that are grouped together to perform an operation.
- A method is always defined inside a class.
- *Method signature* or *method header* is the combination of the method name and the parameter list.

Method definitions have the form

```
method header {
    method body
}
```

Parts of a Method Header

Method Modifiers | Return Type | Method Name | Parentheses

```
public int getEmpNum ()
 {

     return empnum;

 }
```

The above method is an instance method.
The type of result returned is *returnType* where int means the method returns an integer result.

Parts of a Method Header

Method Modifiers | Return Type | Method Name | Parentheses

```
 public void setEmpNum (int num)
  {

      empnum = num;

  }
```

The above method is an instance method.
The type of result returned is *returnType* where void means the method does not return a result.

return Statement

- A method, unless void, returns a value of the specified type to the calling method.
- The return statement is used to immediately quit the method and return a value:

return *expression*;

The type of the return value or expression must match the method's declared return type.

return Statement (cont'd)

- A method can have several return statements; then all but one of them must be inside an if or else (or in a switch):

```
public someType myMethod (...)
{
    ...
    if (...)
        return <expression1>;
    else if (...)
        return <expression2>;
    ...
    return <expression3>;
}
```

return Statement (cont'd)

- A boolean method can return true, false, or the result of a boolean expression:

```
public boolean myMethod (...)
{
   ...
   if (...)
      return true;
   ...
   return n % 2 == 0;
}
```

Calling an instance method

- A method executes when it is called.
- The main method is automatically called when a program starts, but other methods are executed by method call statements
- A method with a non-void return value type can also be invoked as a statement.
- An invocation of an instance method that returns an integer value may use an assignment statement or output statement:

```
int vnum;
vnum = obj1.getEmpNum();
```

or

```
System.out.println ("employee num:" +
   obj1.getEmpNum());
```

Calling an instance method

- An invocation of a `void` instance method is simply a statement:

```
objectName.methodName();
```

- Example:

```
obj1.setEmpNum(123);
```

Passing Parameters to Constructors

- Any expression that has an appropriate data type can serve as a parameter:

```
public static void main (String[] args) {
    Employee emp1 = new Employee();
    Employee emp2 = new Employee(123); // argument
    Employee emp3= new Employee(456, "Ahmad"); // argument
}
```

```
public class Employee {
   public Employee () {  }
   public Employee (int num) {  // parameters
       empnum = num ;}
   public Employee (int num, String name) {
       empnum = num ;
       empname = name;
    }
```

Arguments are Passed by Value

- In Java, all arguments of the primitive data types are *passed by value*, which means that only a copy of an argument's value is passed into a parameter variable.
- Any change to a parameter variable inside a method, has no affect to the original argument.

Passing Parameters : primitive type

- Primitive data types are always passed "<u>by value</u>": the value is copied into the parameter.

```
int x = 123;
emp1.setEmpNum ( x );
```

```
public class Employee
{
   ...
   public void setEmpNum (int num)
   {
      empnum = num;
      ...
   }
}
```

copy

num acts like a local variable in **setEmpNum**

x: **123** — copy → num: **123**

Passing Parameters (cont'd)

```
public class Test
{
   public double square (double x)
   {
      x *= x;
      return x;
   }

   public static void main(String[ ] args)
   {
      Test calc = new Test ();
      double x = 3.0;
      double y = calc.square (x);
      System.out.println (x + "  " + y);
   }
}
```

x here is a copy of the parameter passed to **square**. The copy is changed, but...

... the original **x** is unchanged. Output: **3 9**

Passing Objects as Arguments

- Objects can be passed to methods as arguments.
- Java passes all arguments *by value*.
- When an object is passed as an argument, the value of the reference variable is passed.
- The value of the reference variable is an address or reference to the object in memory.
- When a method receives a reference variable as an argument, it is possible for the method to modify the contents of the object referenced by the variable.

Passing Object As Arguments to Methods

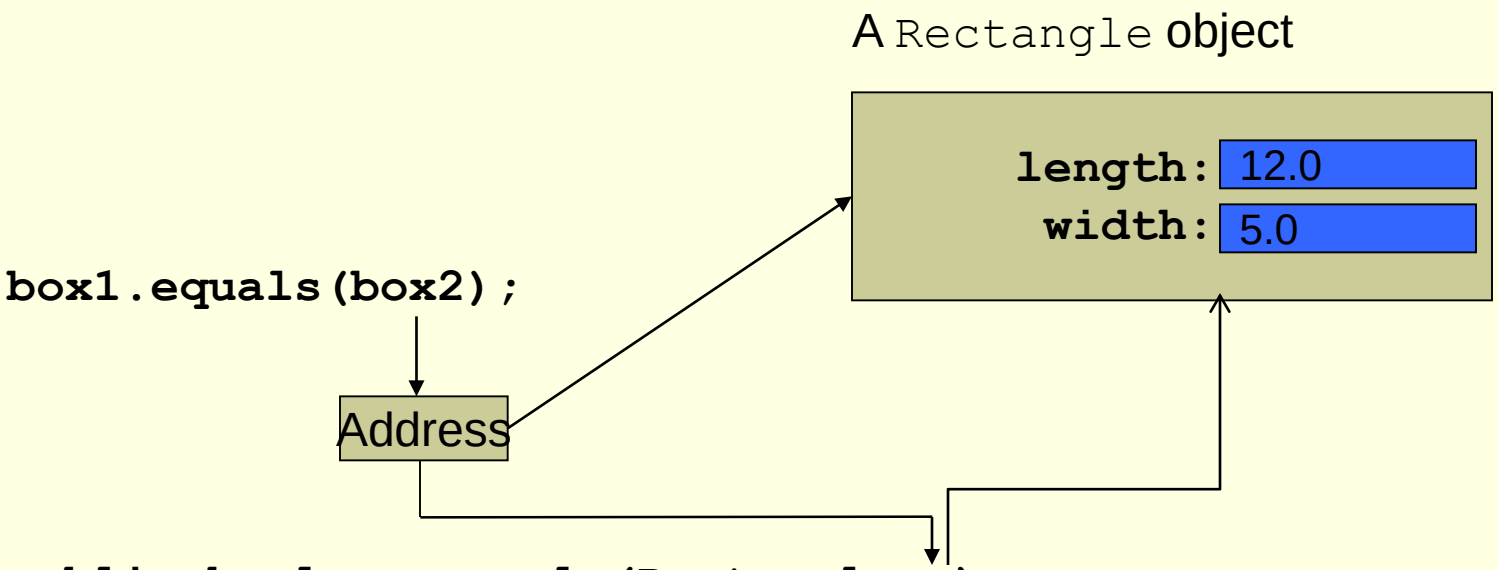
```
public class Rectangle {
  private double width;
  private double length;

  public Rectangle (double w,
                    double l) {
    width = w;
    length = l;
  }

  public void setWidth(double w) {
    width = w;
  }

  public void setLength(double l) {
    length = l;
  }

  public double getWidth() {
    return width;
  }

  public double getLength() {
    return length;
  }

  public double calcArea() {
    return width * length;
  }

  public boolean equals(Rectangle r) {
    return (this.calcArea() == r.calcArea());
  }
}
```

Passing Object As Arguments to Methods

```
1.  public class PassObject  {
2.     public static void main(String[ ] args)  {
3.          Rectangle box1 = new Rectangle(12.0, 5.0);
4.          Rectangle box2 = new Rectangle(10.0, 8.0);
5.          if (box1.equals(box2))
6.             System.out.println ("Yes, it's equals");
7.          else
8.             System.out.println ("No, it's not equal");
9.       }
10.  }
```

Line 3 and 4 creates two instances of Rectangle class.
Line 5 the value of the variable box2 (reference variable of Rectangle) is passed as an argument to the instance method, equals().
In Rectangle class, equals() method has a parameter, r, which is a Rectangle reference variable and return a boolean value, after comparing whether the two objects are equal.

Passing Objects as Arguments




```
public boolean equals(Rectangle r)
{
   return (this.calcArea() == r.calcArea());
}
```

Returning Objects From Methods

```
import java.util.*;

public class PassObject3  {

   public static void main(String[ ] args)  {
      Rectangle box3 = new
                        Rectangle().inputRectangle();
      System.out.println (“The area: “ +
                        box3.calcArea());

   }
```

```
public Rectangle inputRectangle() {
  Scanner inp = new Scanner (System.in);
  double newWidth, newLength;

  System.out.print ("Enter width : ");
  newWidth = inp.nextDouble();
  System.out.print ("Enter length : ");
  newLength = inp.nextDouble();

  return new Rectangle(newWidth, newLength);
  }
}
```

Returning Objects From Methods

- Methods can return references to objects.
- Just as with passing arguments, a copy of the object is not returned, only its address.
- Method return type:

```
public Rectangle inputRectangle ()
{
      …
     return new Rectangle(newWidth, newLength);
}
```

The `toString` Method

- All objects have a `toString` method that returns the class name and a hash of the memory address of the object.
- We can override the default method with our own to print out more useful information.

The `equals` Method

- When the `==` operator is used with reference variables, the memory address of the objects are compared.
- The contents of the objects are not compared.
- All objects have an `equals` method.
- The default operation of the `equals` method is to compare memory addresses of the objects (just like the `==` operator).

The `equals` Method

- The `Rectangle` class has an `equals` method.
  - Write a method that only the addresses of the objects are compared
- Now, instead of using the `==` operator to compare two `Rectangle` objects, we should use the `equals` method.
  - Compare objects by their contents rather than by their memory addresses

Methods That Copy Objects

- There are two ways to copy an object.
  - You cannot use the assignment operator to copy reference types
  - Reference only copy
    - This is simply copying the address of an object into another reference variable.
  - Deep copy (correct)
    - This involves creating a new instance of the class and copying the values from one object into the new object

Copy Constructors

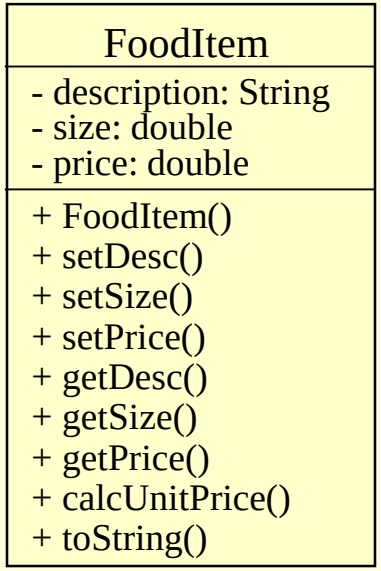
- A copy constructor accepts an existing object of the same class and clones it

```
public Rectangle(Rectangle object2)
{
    this.width = object2.width;
    this.length = object2.length;
}

// Create a rectangle object, rect1
Rectangle rect1 = new Rectangle(5.0, 4.5);
//Create rect2, a copy of rect1
 Rectangle rect2 = new Rectangle(rect1);
```

The FoodItem class

The class diagram that represents a food item at a market

UML class diagram



Methods for Class FoodItem

- For most classes, we need to provide methods that perform tasks similar to the ones below.
- a method to create a new FoodItem object with a specified description, size, and price (*constructor*)
- methods to change the description, size, or price of an existing FoodItem object (*mutators*)
- methods that return the description, size, or price of an existing FoodItem object (*accessors*)
- a method that returns the object state as a string (a **`toString()`** method).

Method Headers for Class FoodItem

Method	*Description*
`public FoodItem(String, double, double)`	Constructor - creates a new object whose three data fields have the values specified by its three arguments.
`public void setDesc(String)`	Mutator - sets the value of data field **description** to the value indicated by its argument. Returns no value.
`public void setSize(double)`	Mutator - sets the value of data field **size** to the value indicated by its argument. Returns no value.

Method Headers for Class FoodItem, cont'd.

Method	*Description*
`public void setPrice(double)`	Mutator - sets the value of data field **price** to the value indicated by its argument. Returns no value.
`public String getDesc()`	Accessor - returns a reference to the string referenced by data field **description**.
`public double getSize()`	Accessor - returns the value of data field **size**.

Method Headers for Class FoodItem, cont'd.

Method	*Description*
`public double getPrice()`	Accessor - returns the value of data field **`price`**.
`public String toString()`	Returns a string representing the state of this object.
`public double calcUnitPrice()`	Returns the unit price (price / size) of this item.

Class FoodItem

```
/*
 * FoodItem.java
 * Represents a food item at a market
 */
public class FoodItem {

  // data fields
  private String description;
  private double size;
  private double price;
```

Class FoodItem, cont'd.

```
// methods
// postcondition: creates a new object with data
//   field values as specified by the arguments
public FoodItem(String desc, double aSize,
                  double aPrice) {
   description = desc;
   size = aSize;
   price = aPrice;
}

// postcondition: sets description to argument
//   value
public void setDesc(String desc) {
   description = desc;
}
```

Class FoodItem, cont'd.

```
// postcondition: sets size to argument value
public void setSize(double aSize) {
   size = aSize;
}

// postcondition: sets price to argument value
public void setPrice(double aPrice) {
   price = aPrice;
}

// postcondition: returns the item description
public String getDesc() {
   return description;
}

// postcondition: returns the item size
public double getSize() {
   return size;
}
```

Class FoodItem, cont'd.

```
  // postcondition: returns the item price
  public double getPrice() {
     return price;
  }

  // postcondition: returns string representing
  //  the item state
  public String toString() {
     return description + ", size : " + size +
            ", price $" + price;
  }

  // postcondition: calculates & returns unit price
  public double calcUnitPrice() {
     return price / size;
  }
}
```

void Methods

- The mutator methods are **`void`** methods.

```
public void setSize(double aSize) {
    size = aSize;
}
```

- Method **`setSize()`** sets the value of data field **`size`** to the value passed to parameter **`aSize`**.
- A **`void`** *method is executed for its effect* because it does not return a value; however, it may change the state of an object or cause some information to be displayed.

The return Statement

- Each **`FoodItem`** method that is not a **`void`** method or a constructor returns a single result.

```
public double getPrice() {
    return price;
}
```

The **return statement** above returns the value of data field **`price`** as the function result.

- Form: **`return`** expression;
- Example: **`return x + y;`**
- Interpretation: The expression after the keyword **`return`** is returned as the method result. Its data type must be assignment compatible with the type of the method result.

toString() Method

- A **`toString()`** method returns a reference to a **`String`** object that represents an object's state.

```
public String toString() {
   return description + ", size : " + size +
          ", price $" + price;
}
```

The expression after **`return`** specifies the characters stored in the **`String`** object returned as the method result. It starts with the characters in **`description`** followed by the characters **`, size :`** followed by characters representing the value of data field **`size`** and **`price`**; e.g.,

`"apples, Macintosh, size: 2.5, price $1.25"`

Calling methods

- Because a **`void`** method does not return a value, a call to a void method is a Java statement all by itself. If **`myCandy`** is type **`FoodItem`**, an example would be

```
myCandy.setDescription("Snickers, large");
```

- Because a non-**`void`** method returns a value, a call to a non-**`void`** method must occur in an expression:

```
"price is $" + myCandy.getPrice()
```

- The result returned by the call to method **`getPrice()`** (type **`double`**) is appended to the string that starts with the characters **`price is $`**

Call to Constructor

```
public FoodItem(String desc, double aSize,
                double aPrice) {
   description = desc;
   size = aSize;
   price = aPrice;
}
```

- A constructor executes when a new object is created and initializes the object's data fields. A call to this constructor always follows the word **new** and has the form:

```
FoodItem myCandy = new FoodItem("Snickers",
                                6.5, 0.55);
```

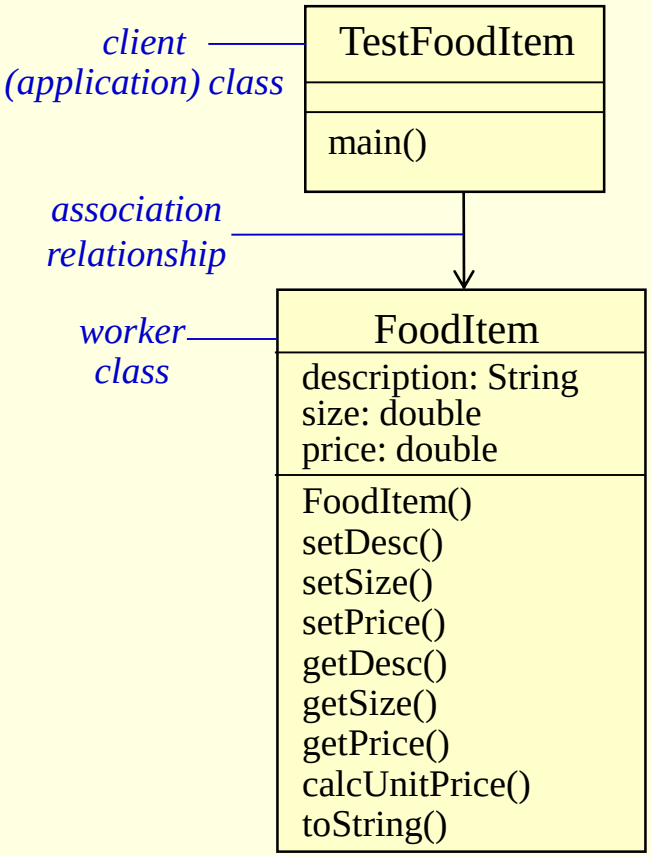
Argument/Parameter Correspondence for `FoodItem("Snickers", 6.5, 0.55)`

Argument	*Parameter*
`"Snickers"`	`desc (references "Snickers")`
`6.5`	`aSize (value is 6.5)`
`0.55`	`aPrice (value is 0.55)`

Primitive type arguments are passed by value which means the method cannot change the argument value.

Testing the class FoodItem

UML class diagram



Program classes and their relationships

Application or Client of FoodItem

```
/*
 * TestFoodItem.java
 * Tests class FoodItem
 */
public class TestFoodItem {

  public static void main(String[] args) {
    FoodItem myCandy = new FoodItem("Snickers", 6.5,
                                          0.55);
    FoodItem mySoup = new FoodItem(
                          "Progresso Minestrone",
                                 16.5, 2.35);
    System.out.println(myCandy.toString());
    System.out.println(" -- unit price is $" +
                       myCandy.calcUnitPrice());
    System.out.println(mySoup.toString());
    System.out.println(" -- unit price is $" +
                       mySoup.calcUnitPrice());
  }
}
```

Sample Run of Class TestFoodItem

TestFoodItem

```
Snickers, size : 6.5, price $0.55
 -- unit price is $0.08461538461538462
Progresso Minestrone, size : 16.5, price $2.35
 -- unit price is $0.14242424242424243
```

The `equals` Method

- The `Rectangle` class has an `equals` method.
- If we try the following:

```
Stock stock1 = new Stock("GMX", 55.3);
Stock stock2 = new Stock("GMX", 55.3);
if (stock1 == stock2) // This is a mistake.
  System.out.println("The objects are the same.");
else
  System.out.println("The objects are not the
  same.");
```

only the addresses of the objects are compared.

Static Variable and Method

Lecture 9

Based on Slides of Dr. Norazah Yusof

Static Modifier

- **The static modifier is used to create variables and methods that will exist independently of any instances created for the class.**
- **In other words, static members exist before you ever make a new instance of a class, and there will be only one copy of the static member regardless of the number of instances of that class.**

Reason for using Static Modifier

1. Need to have a method that always run the same way without having dependency on the instance variable values.
   - Therefore, need to ask the class itself to run the method, and not instance-specific.
2. Need to count all instances instantiated from a particular class.
   - It won't work to keep it as an instance variable within the class whose instances you're tracking, because the count will just be initialized back to a default value with each new instance.
3. Variables and methods marked static belong to the class, rather than to any particular instance.
4. You need only have the class available to be able to invoke a static method or access a static variable.
5. Static variables, too, can be accessed without having an instance of a class. But if there are instances, a static variable of a class will be shared by all instances of that class; there is only one copy.
6. If you want all the instances of a class to share data, use static variables.
   - static variables stored values of a variables in a common memory location

Example 1: Static variable

```
public class Kira {
// Declare and initialize static variable
   static int kaunterKira = 0;

   public Kira() {
      kaunterKira += 1;  // Modify the value in
                         // the constructor
   }
   public static void main (String [] args) {
      new Kira();
      new Kira();
      new Kira();

     System.out.println ("Kiraan Kira ialah " +
   kaunterKira);
   }
}
```

Example 1: Static variable

- **In the preceding code, the static kaunterKira variable is set to 0 when the `Kira` class is first loaded by the JVM, before any `Kira` instances are created!**
- **Whenever a `Kira` instance is created, the `Kira` constructor runs and increments the static kaunterKira variable.**
- **When this code executes, three `Kira` instances are created in main(), and the result is:**

```
Kiraan kira ialah 3
```

Example 2: using instance variable

```
public class Kira {
   int kaunterKira = 0;  // Declare and initialize
                         // instance variable
   public Kira() {
      kaunterKira += 1;  // Modify the value in
                         // the constructor
   }
   public static void main (String [] args) {
      new Kira();
      new Kira();
      new Kira();

      System.out.println ("Kiraan Kira ialah " +
   kaunterKira);
   }
}
```

Example 2: using instance variable

- **When this code executes, it should still create three Kira instances in main() , but the result is…a compiler error!**

```
Kira.java:11: non-static variable kaunterKira cannot
be referenced from a static context
   System.out.println ("Kiraan Kira ialah " +
   kaunterKira);
                                                  ^
 1 error

 Tool completed with exit code 1
```

Static Modifier

- **Things you can mark as static:**
  - **Methods**
  - **Variables**
  - **A class nested within another class, but not within a method**
  - **Initialization blocks**

Static Modifier

- **Things you can't mark as static:**
  - **Constructors (makes no sense; a constructor is used only to create instances)**
  - **Classes (unless they are nested)**
  - **Interfaces**
  - **Method local inner classes (will cover in later chapter)**
  - **Inner class methods and instance variables**
  - **Local variables**

Static nested class

- **A static nested class is simply *a class that's a static member of the enclosing class:***

```
class BigOuter {
  static class Nested { } }
```

- **The static modifier in this case says that the nested class is *a static member of the outer class.***
  - **That means it can be accessed, as with other static members, *without having an instance of the outer class.***

Using Static Nested Classes

- **You use standard syntax to access a static nested class from its enclosing class. The syntax for instantiating a static nested class from a non-enclosing class is a little different from a normal inner class, and looks like this:**

```
class BigOuter {
  static class Nest {
   void go(){System.out.println("hi");}
  } }
class Broom {
   static class B2 {
     void goB2() {System.out.println("hi 2"); }
    }
     public static void main(String[] args) {
       BigOuter.Nest n = new BigOuter.Nest();
       n.go();
       B2 b2 = new B2(); // access the enclosed class
       b2.goB2();
      }}
```

Static variable

- **Static variables store values for the variables in the common memory location.**
- **All object of the same class are affected if one object changes the value of a static variable.**
- **Public static variables are referred to using "dot notation" (if in other classes):**

```
ClassName.staticVar
```

Static variables

```
double area = Math.PI * r * r;
setBackground(Color.BLUE);
c.add(btn, BorderLayout.NORTH);
System.out.println(area);
```

Static final

- A static final can hold a constant shared by all objects of the class:

```
public class RollingDie
{
   private static final double slowDown = 0.97;
   private static final double speedFactor = 0.04;
   ...
```

Reserved words:
static final

Static Methods

- **Static methods can access and manipulate a class's static fields.**
- **Static methods <u>cannot</u> access non-static fields or call non-static methods of the class.**
- **Static methods are called using "dot notation":** `ClassName.staticMethod(...)`

```
double x = Math.random();
double y = Math.sqrt (x);
System.exit();
```

Lab 3 Exercise 1

Now, let's do Question 1(v) on page 63 :

i. **Determine the output of Program 3.6. What type of argument passed by *max*?**

Example of Static Methods

pass the value of i

pass the value of j

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

- **The above program contains the `main` method and the `max` method (which is a static method) which demonstrates calling a method `max` to return the largest of the `int` values.**
- **In this example, the `main` method invokes `max(i,j)`, which is defined in the same class with the `main` method.**

Example of Static Methods

pass the value of i

pass the value of j

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

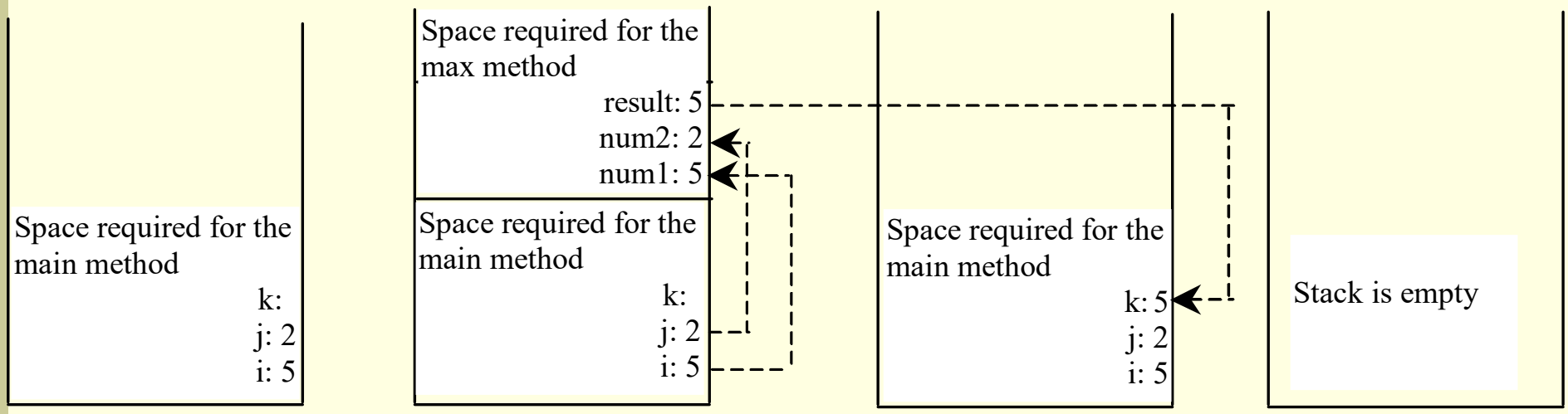
```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

- **When the max method is invoked, variable `i`'s value 5 is passed to `num1`, and variable `j`'s value 2 is passed to `num2` in the `max` method.**
- **The flow of control transfers to the `max` method.**
- **The `max` method is executed. When the `return` statement in the `max` method is executed, the `max` method `returns` the control to its caller.**

Call Stacks

- Each time a method is invoked, the system stores parameters and variables in an area of memory, known as stack, which stores elements in last-in-first-out fashion.
- When a method calls another method, the caller's stack space is kept intact, and new space is created to handle the new method call.
- When a method finishes it work and returns to its caller, its associated space is released.

Call Stacks



The main method is invoked.

The max method is invoked.

The max method is finished and the return value is sent to k.

The main method is finished.

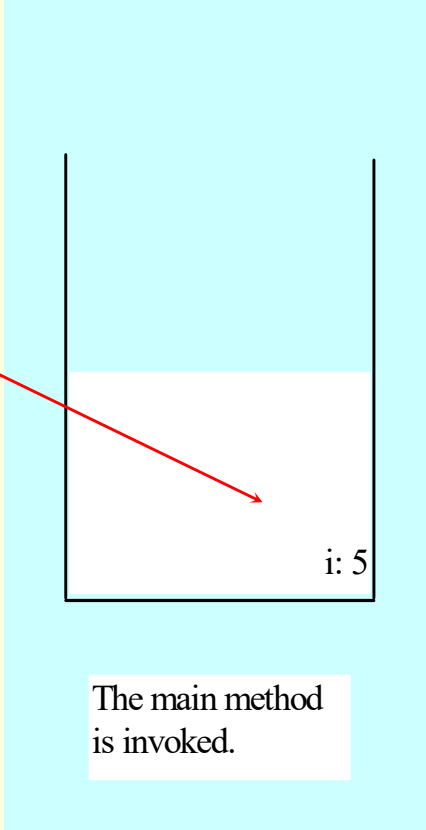
animation

Trace Call Stack

i is declared and initialized

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



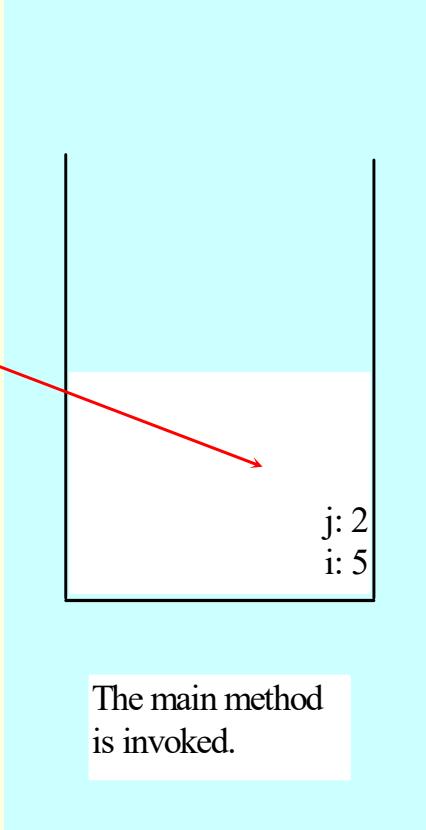
animation

Trace Call Stack

j is declared and initialized

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



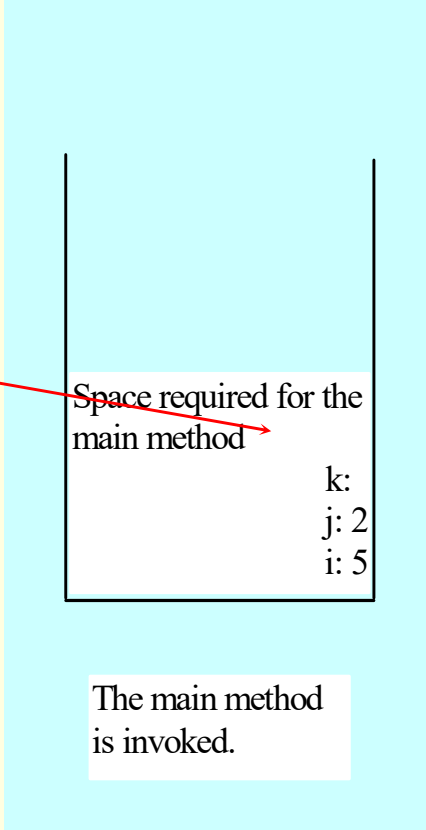
animation

Trace Call Stack

Declare k

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



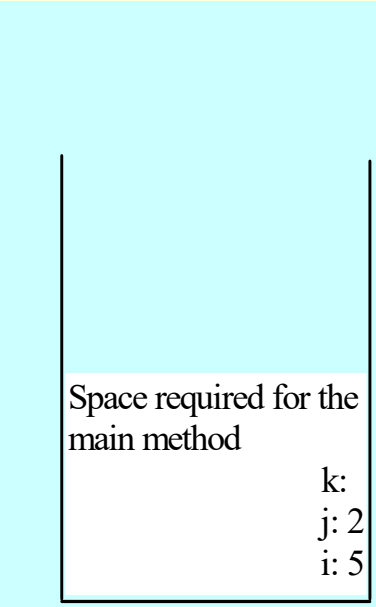
animation

Trace Call Stack

Invoke max(i, j)

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



The main method is invoked.

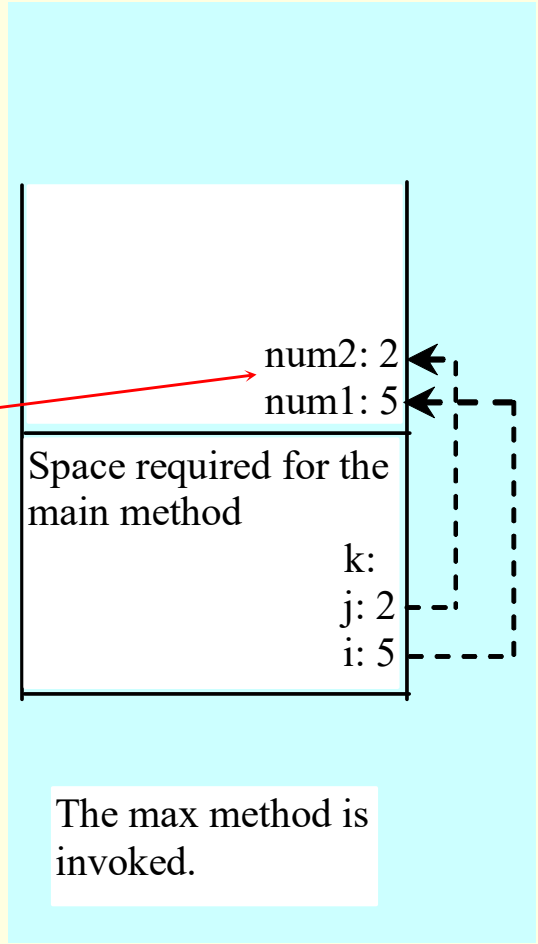
animation

Trace Call Stack

pass the values of i and j to num1 and num2

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



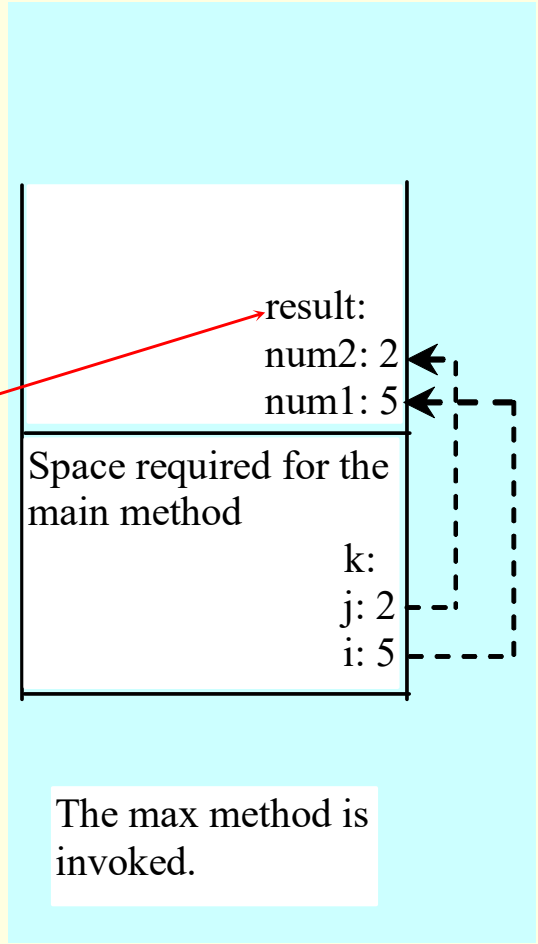
animation

Trace Call Stack

pass the values of i and j to num1 and num2

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



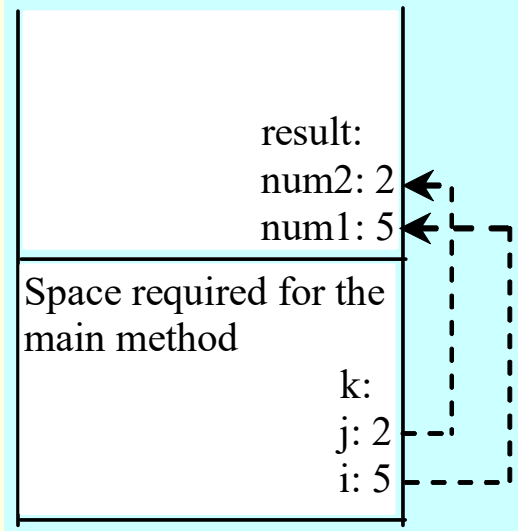
animation

Trace Call Stack

(num1 > num2) is true

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



The max method is invoked.

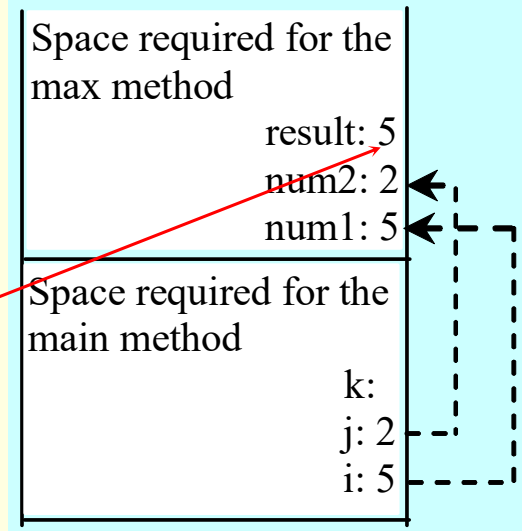
animation

Trace Call Stack

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2)
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

Assign num1 to result



The max method is invoked.

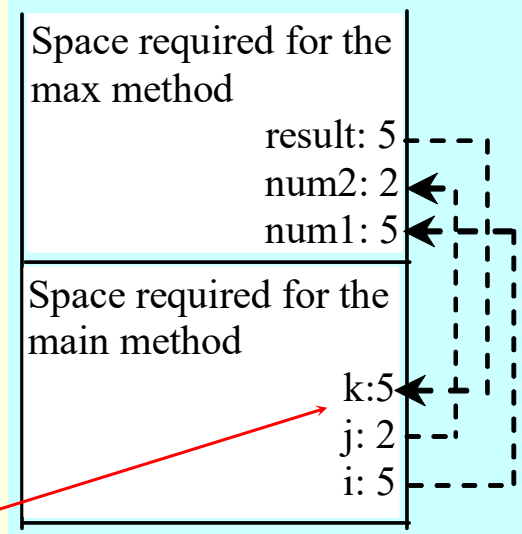
animation

Trace Call Stack

Return result and assign it to k

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



The max method is invoked.

animation

Trace Call Stack

Execute print statement

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

Space required for the main method

k:5
j: 2
i: 5

The main method is invoked.

Static method cannot access an instance variable

- Accessing `size` (instance variable) from inside `doMore` static method

```
class Foo

int size = 42;
static void doMore( ){
  int x = size;
}
```

static method cannot access an instance (non-static) variable

Static method cannot access a non-static method

- Accessing `go` (non-static method) from inside `doMore` static method

```
class Bar

void go ( );
static void doMore( ){
    go( );
}
```

static method cannot access a non-static method

Static method can access a static method or variable

- Accessing `woo` (static method) and `count` (static variable) from inside `doMore` static method

```
class Baz

static int count;
static void woo( ){ }
static void doMore( ){
    woo( );
    int x = count;
}
```

static method *can* access a static method or variable

Scope of Local Variables

A local variable: a variable defined inside a method.

Scope: the part of the program where the variable can be referenced.

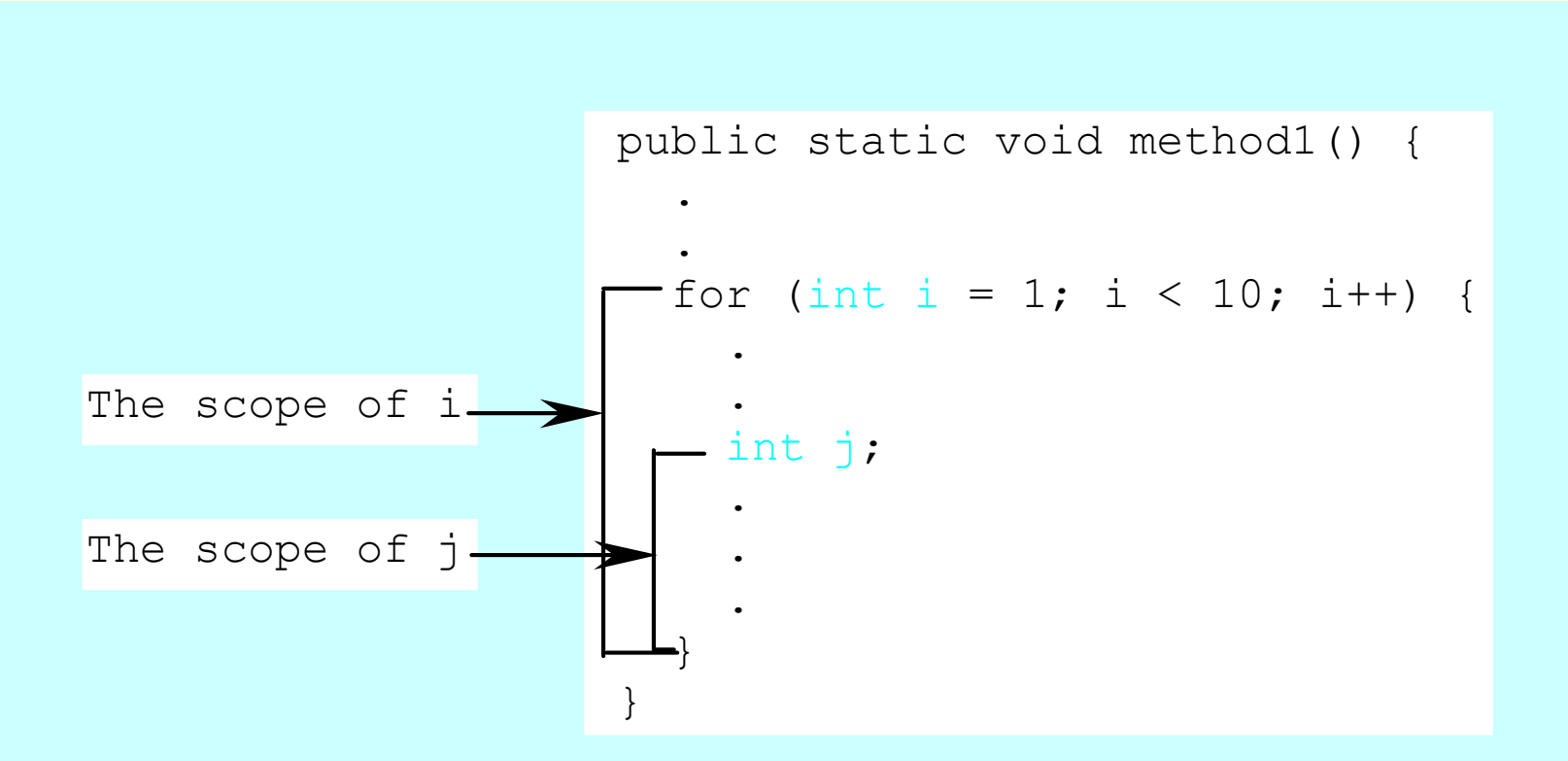
The scope of a local variable starts from its declaration and continues to the end of the block that contains the variable.

Scope of Local Variables, cont.

You can declare a local variable with the same name multiple times in different non-nesting blocks in a method, but you cannot declare a local variable twice in nested blocks.

Scope of Local Variables, cont.

A variable declared in the initial action part of a for loop header has its scope in the entire loop. But a variable declared inside a for loop body has its scope limited in the loop body from its declaration and to the end of the block that contains the variable.



Scope of Local Variables, cont.

It is fine to declare i in two non-nesting blocks

```
public static void method1() {
  int x = 1;
  int y = 1;

  for (int i = 1; i < 10; i++) {
    x += i;
  }

  for (int i = 1; i < 10; i++) {
    y += i;
  }
}
```

It is wrong to declare i in two nesting blocks

```
public static void method2() {

  int i = 1;
  int sum = 0;

  for (int i = 1; i < 10; i++)
    sum += i;
  }

}
```

Scope of Local Variables, cont.

```
// Fine with no errors
public static void correctMethod() {
  int x = 1;
  int y = 1;
  // i is declared
  for (int i = 1; i < 10; i++) {
    x += i;
  }
  // i is declared again
  for (int i = 1; i < 10; i++) {
    y += i;
  }
}
```

Scope of Local Variables, cont.

```
// With errors
public static void incorrectMethod() {
  int x = 1;
  int y = 1;
  for (int i = 1; i < 10; i++) {
    int x = 0;
    x += i;
  }
}
```

Method Overloading

- Method overloading is two or more methods have the same name, but different parameter lists within one class.
- The Java compiler determines which method is used based on the method signature.
- Overloaded methods must have different parameter lists. You cannot overload methods based on different modifiers or return type.

Overloading Methods

Listing 5.3 Overloading the `max` Method

```
public static int max(int num1, int num2) {
  if (num1 > num2)
    return num1;
  else
    return num2;
}

public static double max(double num1, double num2) {
  if (num1 > num2)
    return num1;
  else
    return num2;
}

public static double max(double num1, double num2, double num3) {
  return max(max(num1, num2), num3);
}
```

Ambiguous Invocation

- Sometimes there may be two or more possible matches for an invocation of a method, but the compiler cannot determine the most specific match.
- This is referred to as *ambiguous invocation*. Ambiguous invocation is a compilation error.

Ambiguous Invocation

```
public class AmbiguousOverloading {
  public static void main(String[] args) {
    System.out.println(max(1, 2));
  }

  public static double max(int num1, double num2) {
    if (num1 > num2)
      return num1;
    else
      return num2;
  }

  public static double max(double num1, int num2) {
    if (num1 > num2)
      return num1;
    else
      return num2;
  }
}
```

Lab 3 Exercise 1

Answer Question 1 on page 61 :

i. Determine the output of Program 3.1 & 3.2. Does the values of *i* and *j* for *t1*, *t2* and *t3* are same? Why?

ii. Determine the output of Program 3.3. What is the value of the first, second and third of *kaunterTest2*? Why this happen?

iii. Determine the output of Program 3.4. What is the value of the first, second and third of *kaunterTest3*? Why this happen?

iv. Determine the output of Program 3.5. What is the value of *kaunterTest4*? Why this happen?

Answer Question 2 on page 63

Answer : Lab 3 Exercise 1

Answer Question 1 on page 61 :

i. Determine the output of Program 3.1 & 3.2.

t1's static i= 1 and instance j=1
t2's static i= 2 and instance j=1
t3's static i= 3 and instance j=1

ii. Determine the output of Program 3.3.

Kiraan Test2 ialah 1 2 3

iii. Determine the output of Program 3.4.

Kiraan Test3 ialah 1 1 1

iv. Determine the output of Program 3.5.

Kiraan Kira ialah 25

Answer Question 3 on page 64 :

```
System.out.print("Year of start working=" );
startYear = kb.nextInt();
temp = calcYears(thisYear, startYear);
System.out.println("You have been worked for=" + temp);
```